KHUDA

Computer Vision

2주차

딥러닝 기초와 파이토치

트랙장 : 이승준

Contents

01 Recap

02 Optimizer

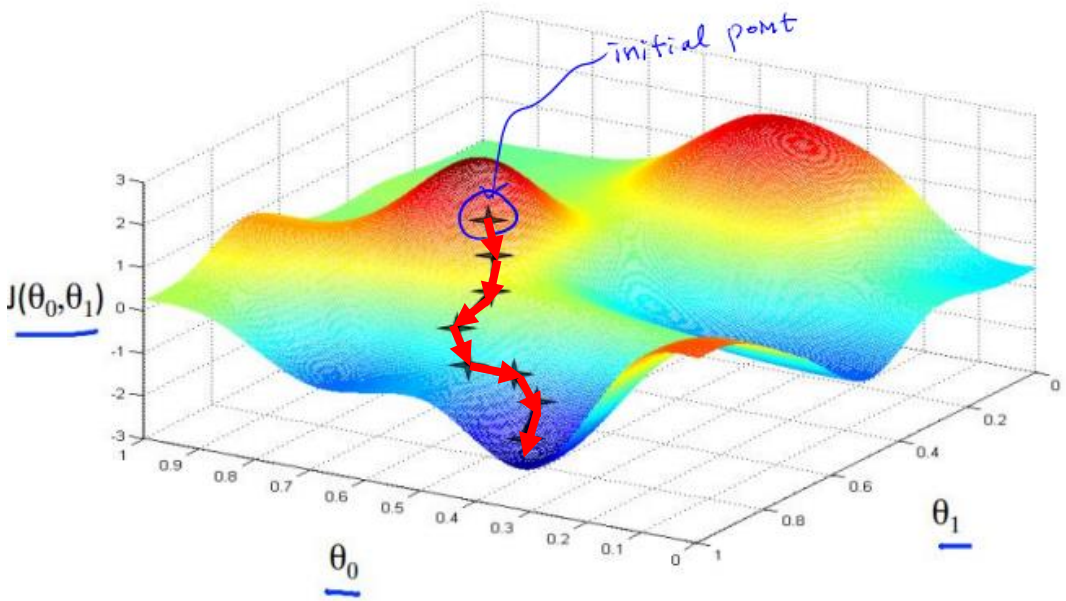
03 Stable Gradient

04 Image

05 Pytorch

Recap

How to Train Model – Gradient Descent



$$new\ \theta = current\ \theta - \alpha \nabla J(\theta)$$

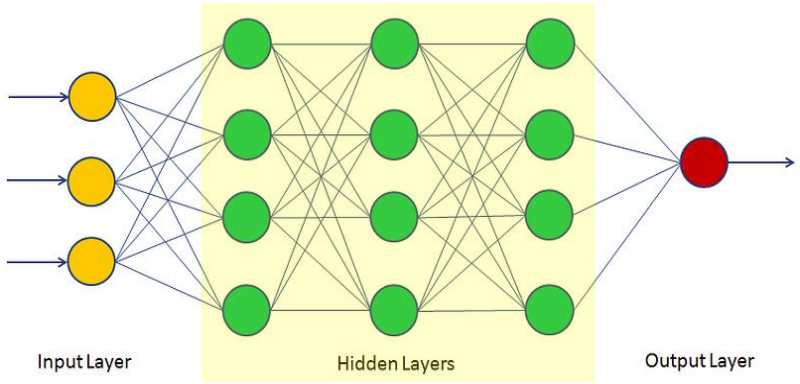
비용 함수라는 산의 정상에서 시작해서
한 step 한 step 거쳐가면서 비용함수를 줄여나가는 방향으로
가중치를 수정하면서 산을 내려오겠다

1. 현재 모델에 특정 데이터를 넣고 Output을 만들어낸다
2. 현재 모델이 만든 Output과 Ground Truth를 비교해서 Error를 계산한다
3. 계산된 Error를 바탕으로 모든 가중치에 대해, Error를 줄이는 방향을 찾아낸다
4. Error를 줄이는 방향으로 learning rate 만큼 가중치를 갱신한다

Activation Function

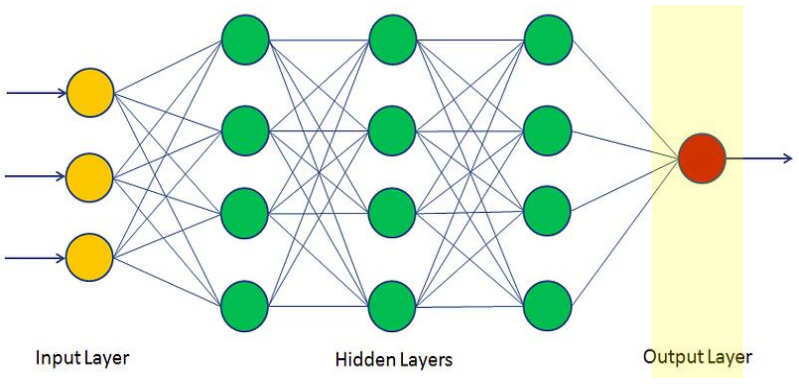
두 가지 관점에서 본 Activation Function

Hidden Layer에 속하는 Activation Function의 관점



Activation Function은 뉴런의 가중합을 입력으로 받아,
이를 비선형적으로 변환함으로써 신경망이
복잡한 패턴과 비선형 함수를 근사할 수 있게 해주는 함수

Output Layer에 속하는 Activation Function의 관점



우리가 원하는 Task에 맞는 Output을 뽑아내기 위해
노드에 들어온 값들을 적절히 변환하여 출력하는 역할을 함

Binary Cross-Entropy Loss

Binary Cross-Entropy Loss

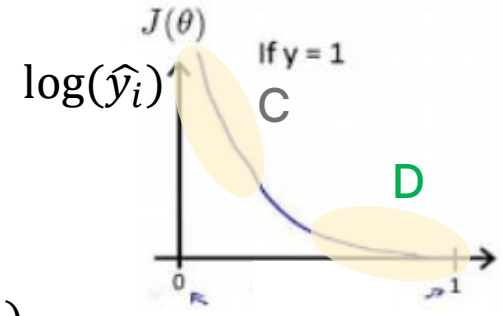
$$BCE = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

예측값이 확률인 경우에 사용하는 Loss로,
이진 분류인 경우 2개 class에 대한 확률이 나오는데, 각 class별로 잘못한 만큼 penalty를 가짐

y_i = **Ground Truth (0 / 1)**
$\hat{y}_i$ = **모델의 예측 (0 ~ 1 확률)**

Case A) 정답이 0일 때, 0에 가깝게 말해서 맞춘 경우 -> loss가 작아야 함
Case B) 정답이 0일 때, 1에 가깝게 말해서 틀린 경우 -> loss가 커야 함
Case C) 정답이 1일 때, 0에 가깝게 말해서 틀린 경우 -> loss가 커야 함
Case D) 정답이 1일 때, 1에 가깝게 말해서 틀린 경우 -> loss가 작아야 함

정답이 0인데, 내 모델이 0에 가깝게 말했을지언정 딱 0으로 예측한 것이 아니니까 loss는 발생해야 함.
하지만 과실을 따져봤을 때 loss의 비중이 달라져야한다는 것

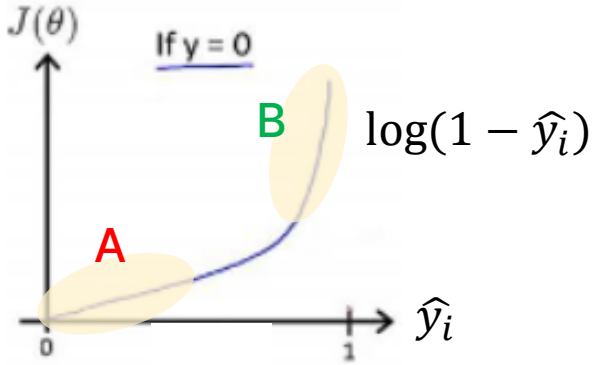


데이터 하나의 Loss

$$[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

정답이 0일 때 $(y_i = 0)$

$$[\cancel{y_i \log(\hat{y}_i)}^{\,0} + (1-\cancel{y_i}^{\,0})\log(1-\hat{y}_i)] \longrightarrow \log(1-\hat{y}_i) \longrightarrow$$



Loss를 줄이는 가중치 갱신 방향 찾기

Output Layer와 연결된 가중치에 필요한 계산

$$w_5 \rightarrow z_3 \rightarrow o_1 \rightarrow E_{total}$$

$$\frac{\partial E_{total}}{\partial w_5} = \frac{\partial E_{total}}{\partial o_1} \times \boxed{\frac{\partial o_1}{\partial z_3}} \times \frac{\partial z_3}{\partial w_5}$$

1층 Hidden Layer에 연결된 가중치에 필요한 계산

$$w_1 \rightarrow z_1 \rightarrow h_1 \rightarrow w_5 \rightarrow z_3 \rightarrow o_1 \rightarrow E_{total}$$

$$w_1 \rightarrow z_1 \rightarrow h_1 \rightarrow w_7 \rightarrow z_4 \rightarrow o_2 \rightarrow E_{total}$$

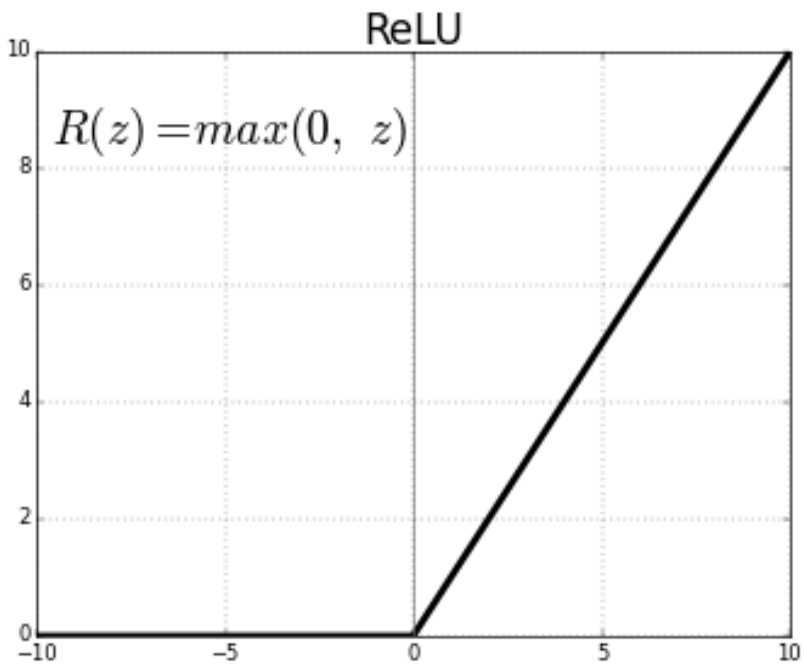
$$\frac{\partial E_{total}}{\partial w_1} = \left(\frac{\partial E_{o1}}{\partial o_1} \times \boxed{\frac{\partial o_1}{\partial z_3}} \times \frac{\partial z_3}{\partial h_1} + \frac{\partial E_{o2}}{\partial o_2} \times \boxed{\frac{\partial o_2}{\partial z_4}} \times \frac{\partial z_4}{\partial h_1} \right) \times \boxed{\frac{\partial h_1}{\partial z_1}} \times \frac{\partial z_1}{\partial w_1}$$

빨간 box는 activation function의 미분임

층이 쌓이면 쌓일수록 가중치를 갱신하기 위해 activation function의 미분이 여러 번 곱해진다

Activation Function

ReLU Function



양수에서는 항상 기울기를 1로, 음수에서는 0으로 계산하는 ReLU가 등장함
Sigmoid, tanh 들의 Gradient 소실 문제를 하였으며, 계산도 쉬워서 학습 속도 및 수렴 속도도 매우 빨라짐

물론, 음수에서는 0이니까 기울기를 못 전달하는거 아닌가요? 라고 말할 수 있으나
애초에 Gradient를 잘 전달해서 음수가 되는 경우가 거의 발생하지 않음 / 또한 0 이하에서 일부를 죽이기 때문에 오히려 Overfitting 완화

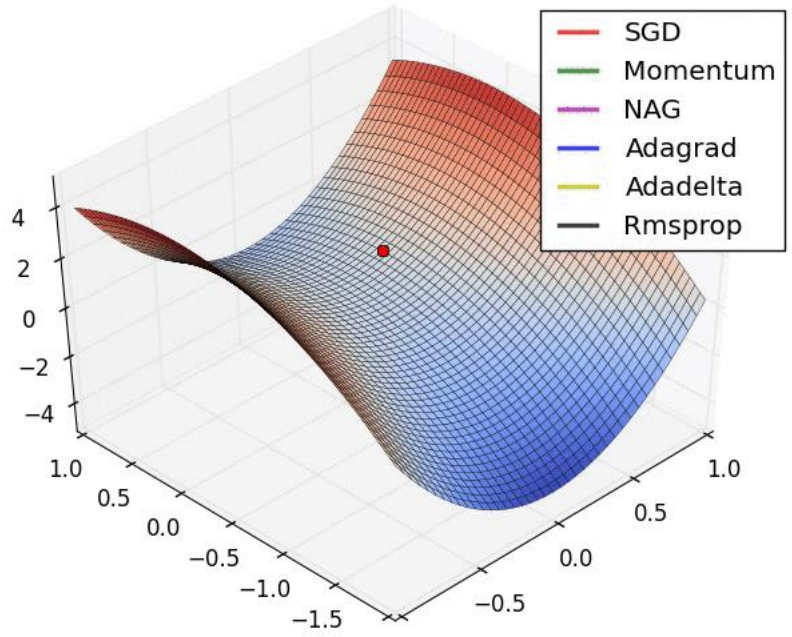
Optimizer

Optimizer

딥러닝 모델은 최대한 틀리지 않는 방향으로 학습해야 함

모델이 얼마나 틀리는지를 찾는 함수가 Loss Function이다.
즉, Loss Function의 최솟값을 찾는 것을 학습 목표로 한다.

최소값을 찾아가는 행위를 Optimization이라고 표현하고, 이를 수행하는 최적화 알고리즘을 Optimizer라고 한다.



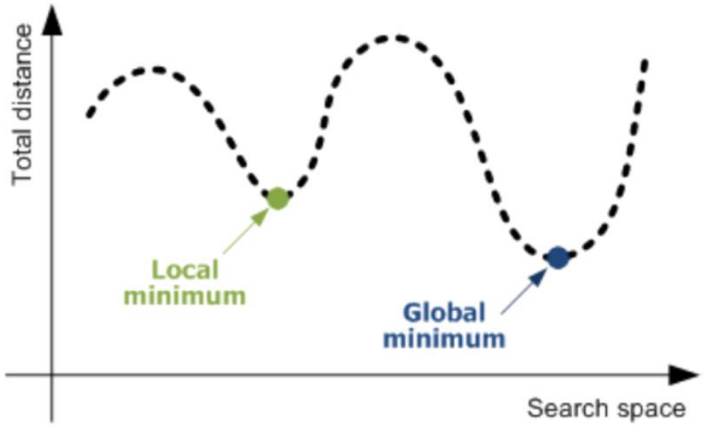
중요 2. Optimizer 종류마다 parameter 갱신 식이 조금씩 달라짐

$$new\ \theta = current\ \theta - \underline{\alpha}\nabla J(\theta)$$

중요 1. 모든 Optimizer에는 Learning Rate가 있다

Optimizer

Optimizer의 목표는 Loss Function의 Global Minimum을 찾아내는 것

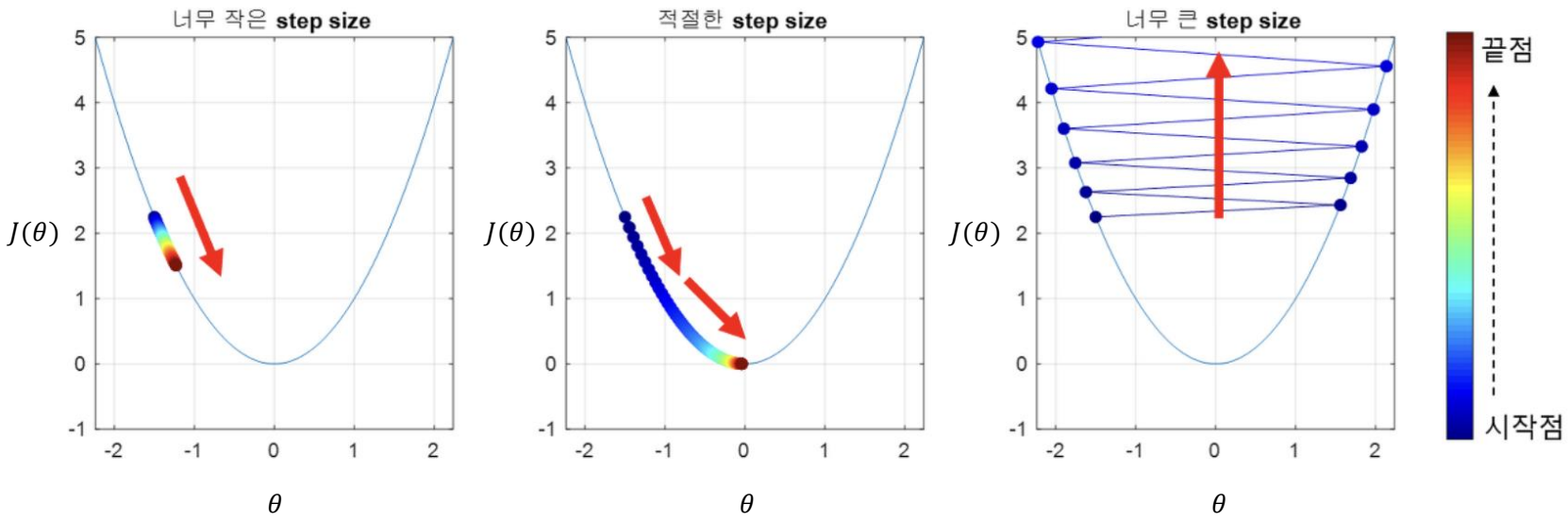


어떤 Optimizer를 쓰는지 / 어느정도의 Learning Rate로 학습하는지

다양한 환경에 따라서 모델이 Global Minimum을 찾을 수도 있고, Local Minimum에 갇힐 수도 있는 것

Learning Rate

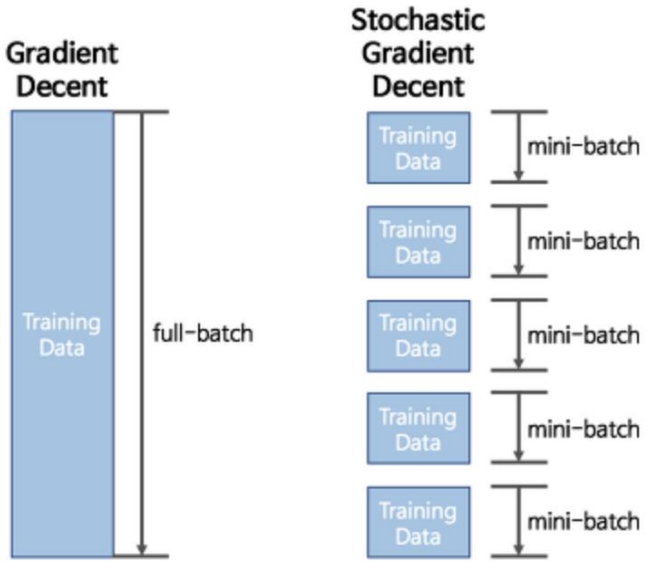
적절한 Learning Rate를 찾는 것이 왜 중요한가?



만약, Learning Rate가 너무 작으면 모델이 Global Minimum에 도달하지 못하고 멈추게 됨

반대로, Learning Rate가 너무 크면 모델이 발산해서 Loss Function이 줄어들지 않게 됨

Optimizer의 종류

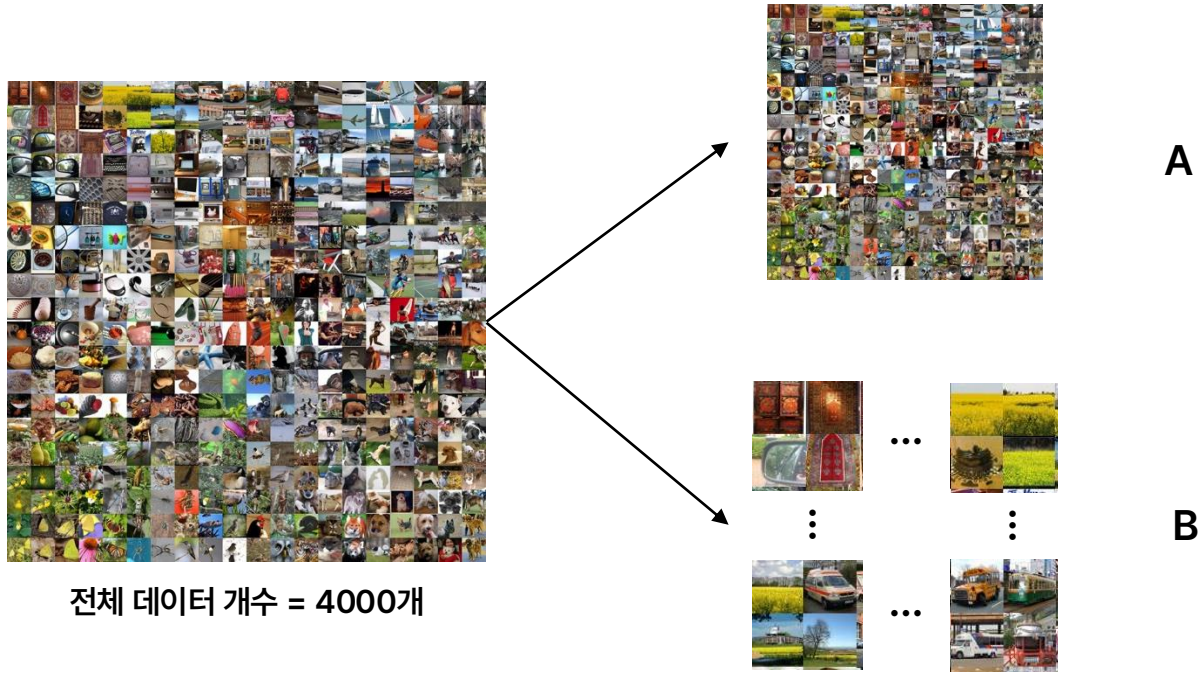


(Full Batch) Gradient Descent – 모든 데이터를 다 사용해서 Loss를 계산하고, 모델의 파라미터를 갱신

On-line Gradient Descent – 데이터를 하나 사용해서 Loss를 계산하고, 모델의 파라미터를 갱신

Mini-Batch Gradient Descent – 모든 데이터 중 일부를 사용해서 Loss를 계산하고, 모델의 파라미터를 갱신

+) 용어 정리



전체 데이터 개수 = 4000개

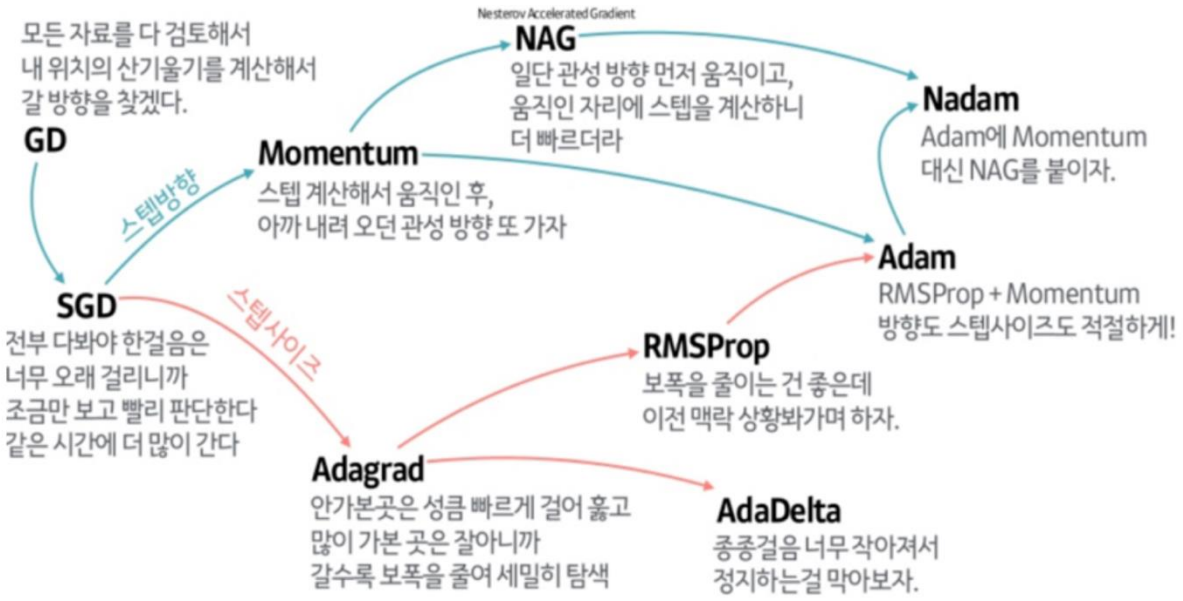
	A	B
Batch size		
Iteration		
(10 Epoch 기준) Iteration		

Epoch – 전체 데이터셋에 대해 한번 학습을 완료하는 행위(시점)

Batch size – 하나의 batch(묶음)에 있는 학습 샘플의 수

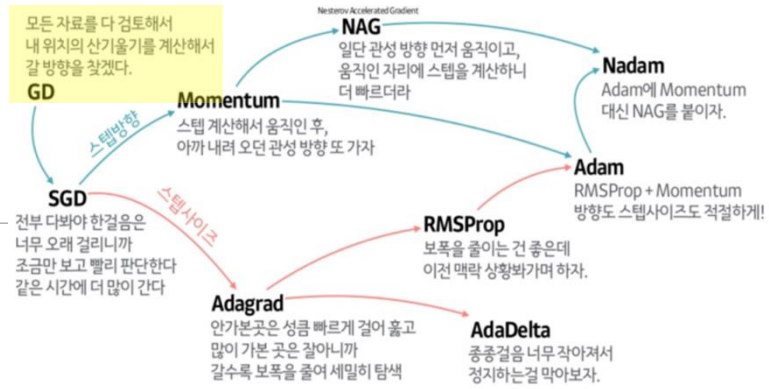
Iteration – 한 Epoch를 완료하기 위해 실행하는 횟수 = 전체 학습 샘플 개수 / Batch size

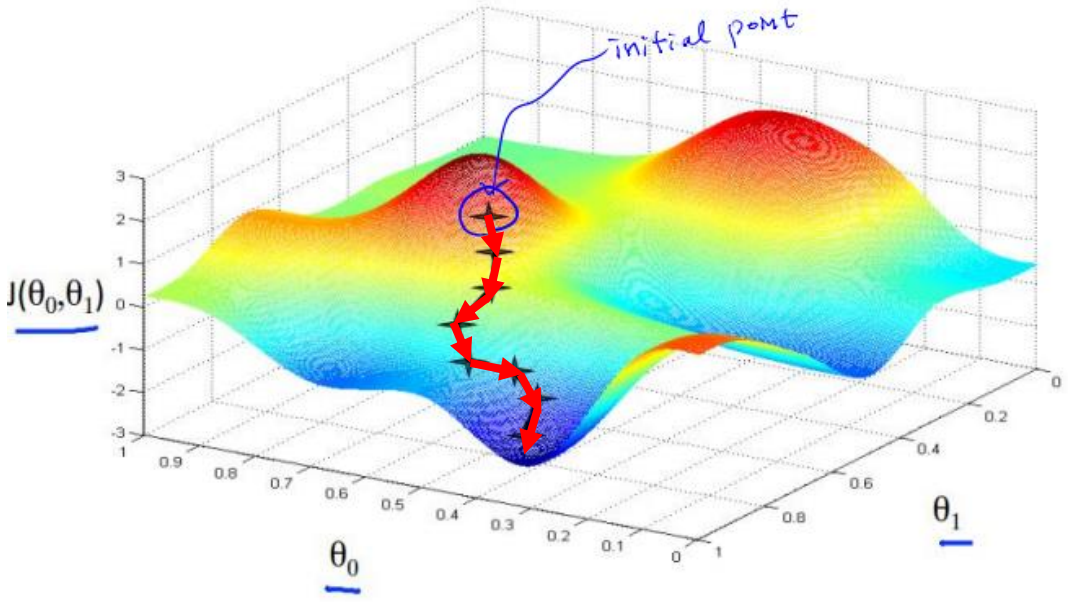
Optimizer의 종류

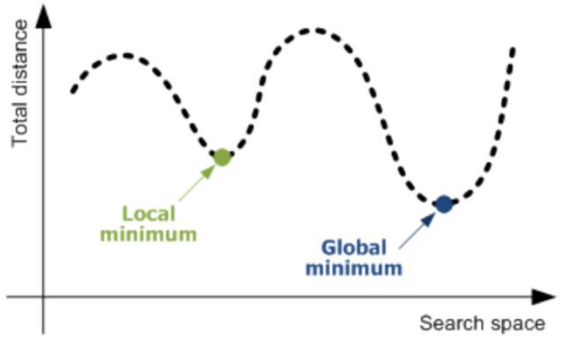


다양한 Optimzer들이 존재함

Gradient Descent



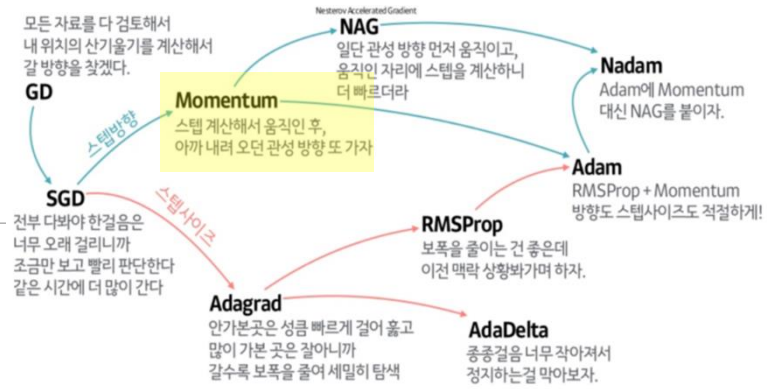




Local Minimum에 빠져서 탈출을 잘 못함

$$new\ \boldsymbol{\theta} = \boldsymbol{\theta} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$$

모멘텀 (Momentum)

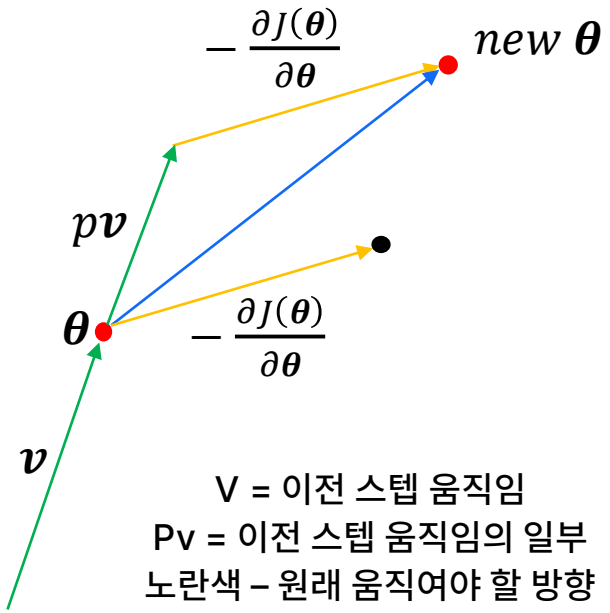


현재 방향을 계산해서 움직이기 전에, 이전 방향으로 조금 더 가는 방법

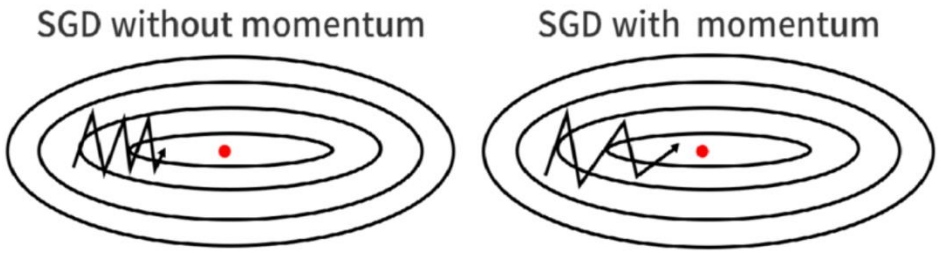
$$new\ \boldsymbol{\theta} = \boldsymbol{\theta} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \Rightarrow new\ \boldsymbol{\theta} = \boldsymbol{\theta} + new\ \boldsymbol{v} \Rightarrow new\ \boldsymbol{\theta} = \boldsymbol{\theta} + p\boldsymbol{v} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$$

$$(new\ \boldsymbol{v} = p\boldsymbol{v} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}})$$

v = 이전 step에서 움직인 방향
p = 이전 방향으로 얼마나 움직일 것인지

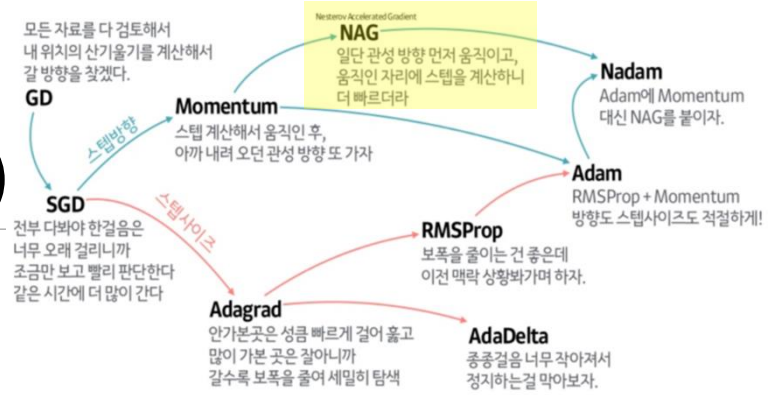


V = 이전 스텝 움직임
Pv = 이전 스텝 움직임의 일부
노란색 – 원래 움직여야 할 방향
파란색 – 모멘텀이 적용된 움직임



지그재그가 줄어들어서, 훨씬 수렴도 빨라지고
pv만큼 더 가는거니까 Local optimum 탈출이 쉬움

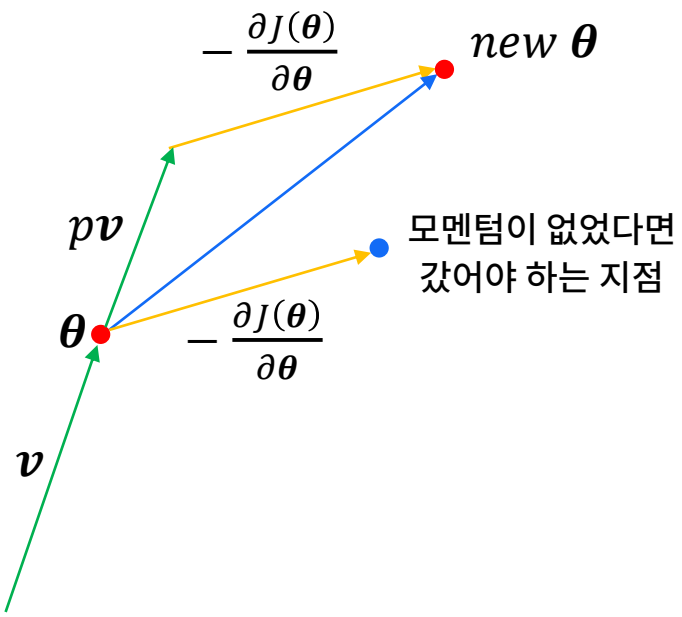
Nesterov Accelerated Momentum (네스테로프 모멘텀)



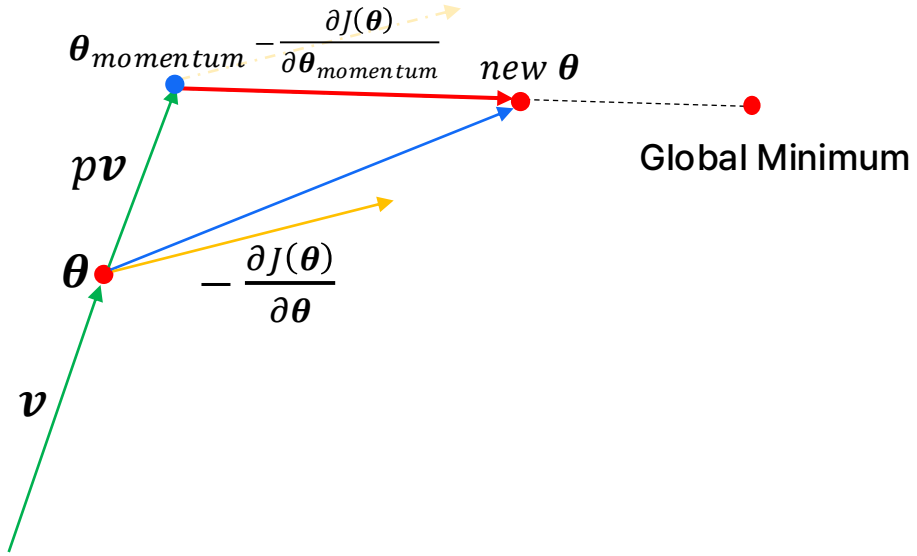
Momemtum 움직임 이후의 세타 방향을 계산해서, 그 방향으로 바로 움직이는 방법

$$new\ \boldsymbol{\theta} = \boldsymbol{\theta} + p\boldsymbol{v} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}} \quad \Rightarrow \quad new\ \boldsymbol{\theta} = \boldsymbol{\theta} + p\boldsymbol{v} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_{momentum}}$$

$\boldsymbol{\theta}_{momentum}$
Momentum 이후의 세타

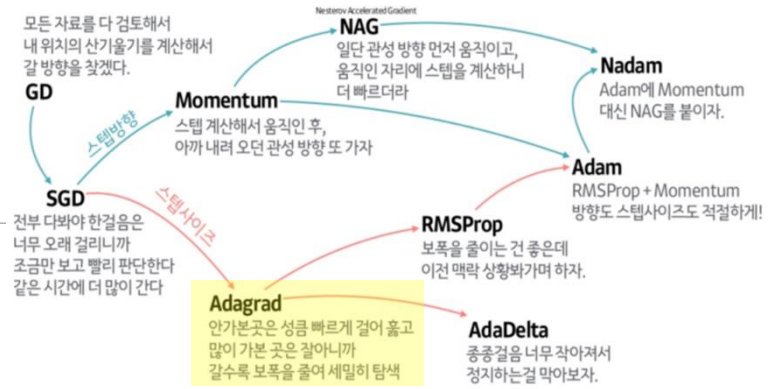


일반 모멘텀



네스테로프 모멘텀

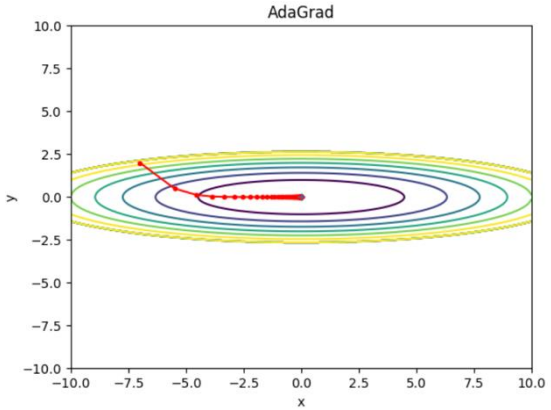
Adagrad (Adaptive Gradient Descent)



모든 가중치에 대해 서로 다른 비율을 적용해서 움직임
(많이 갱신한 = 잘 아는 방향은 조금씩 자세히 움직이고 / 덜 갱신한 = 잘 모르는 방향은 많이 훅 움직임)
복습 = 이미 아니까 더 자세히 들여다보는 것 / 예습 = 잘 모르니까 일단 한번 훑어보는 것 (성큼성큼)

🙇‍♀️ 아래 수식은 이해를 위해 적은 것이고, 실제 수식은 따로 있습니다. (학습률은 그대론데, 학습률 뒤에 가중치마다 다른 새로운 파라미터가 곱해짐)
아무튼 가중치마다 움직이는 비율이 다르다 !! (원래는 같다)

$$h \leftarrow h + \frac{\partial L}{\partial W} \odot \frac{\partial L}{\partial W}$$
$$W \leftarrow W - \eta \frac{1}{\sqrt{h}} \frac{\partial L}{\partial W}$$



$$new\ \boldsymbol{\theta} = \boldsymbol{\theta} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$$

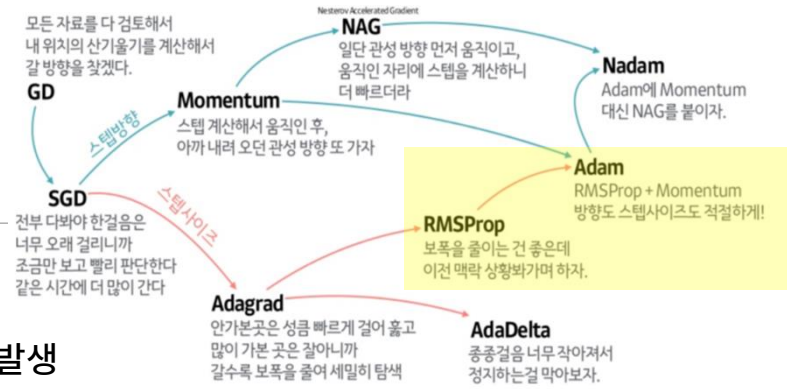
$$new\ \boldsymbol{\theta}_0 = \boldsymbol{\theta}_0 - \alpha_0 \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_0}$$

$$new\ \boldsymbol{\theta}_1 = \boldsymbol{\theta}_1 - \alpha_1 \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_1}$$

$$\vdots$$

$$new\ \boldsymbol{\theta}_n = \boldsymbol{\theta}_n - \alpha_n \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_n}$$

RMSProp (Root Mean Square Propagation)

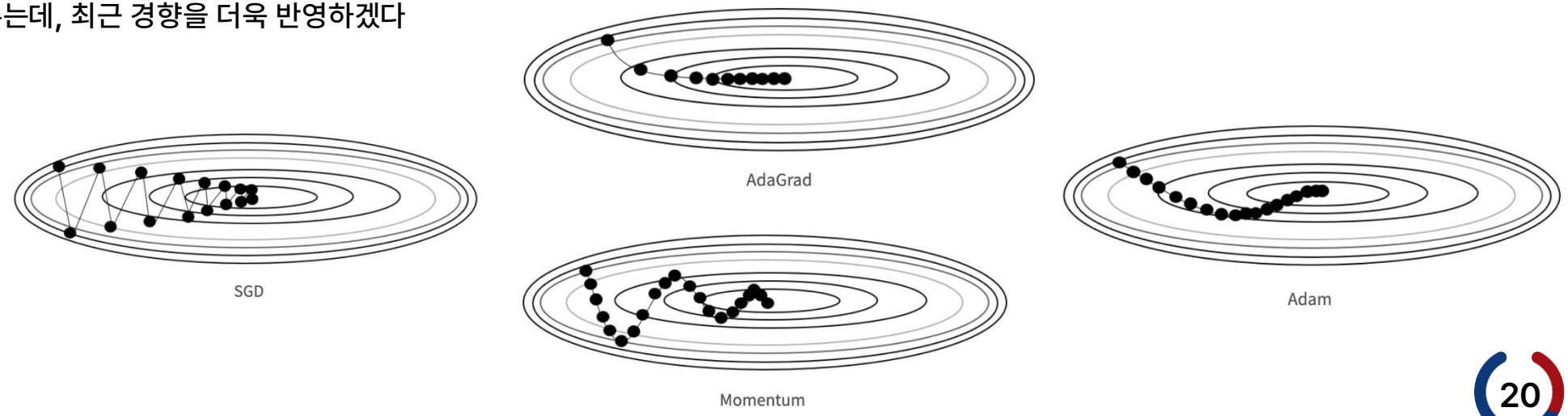


AdaGrad의 단점을 보완한 기법으로, AdaGrad는 과거의 기울기를 제곱하여 더하는 방식이므로
학습을 진행함에 따라 갱신 정도가 약해짐. -> 무한히 학습을 한다면, 갱신량이 거의 0이 되어 학습이 안되는 경우 발생

따라서, RMSProp은 과거의 기울기들을 똑같이 더하는게 아니라, 먼 과거의 기울기는 조금 반영하고, 최신 기울기를 많이 반영

Adam (Adaptive Moment Estimation)

RMSProp + 모멘텀
즉, 스무딩을 적용해서 더 밀어주는데, 최근 경향을 더욱 반영하겠다

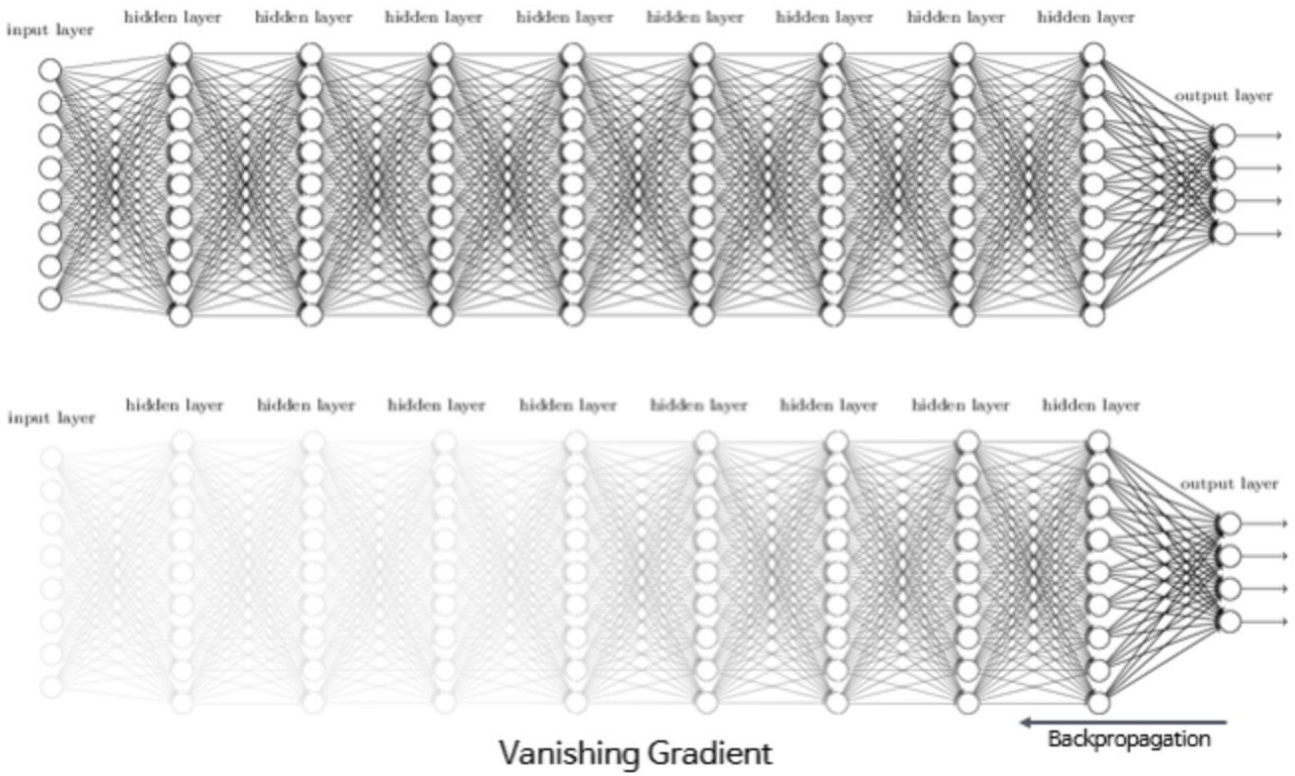


Stable Gradient

Vanishing Gradient Problem

Layer를 많이 쌓을수록 표현력이 증가하기 때문에 학습이 잘 될 것으로 생각할 수 있지만, 반대로 Layer가 많아서 학습이 잘 되지 않을 수 있음

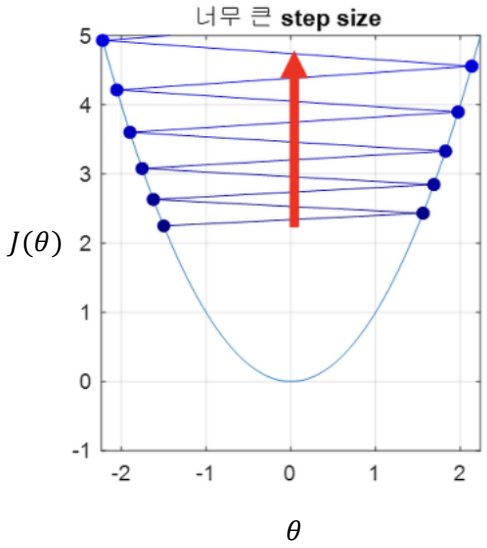
이는 층이 깊으면 깊을수록 출력층에서 오는 Gradient의 값이 매우 작아지기 때문이고, 이를 Vanishing Gradient 현상이라고 부름
발생하는 이유는 지난번 본 것 처럼, activation function의 미분이 여러번 곱해지면서 점차 값이 희미해지기 때문임



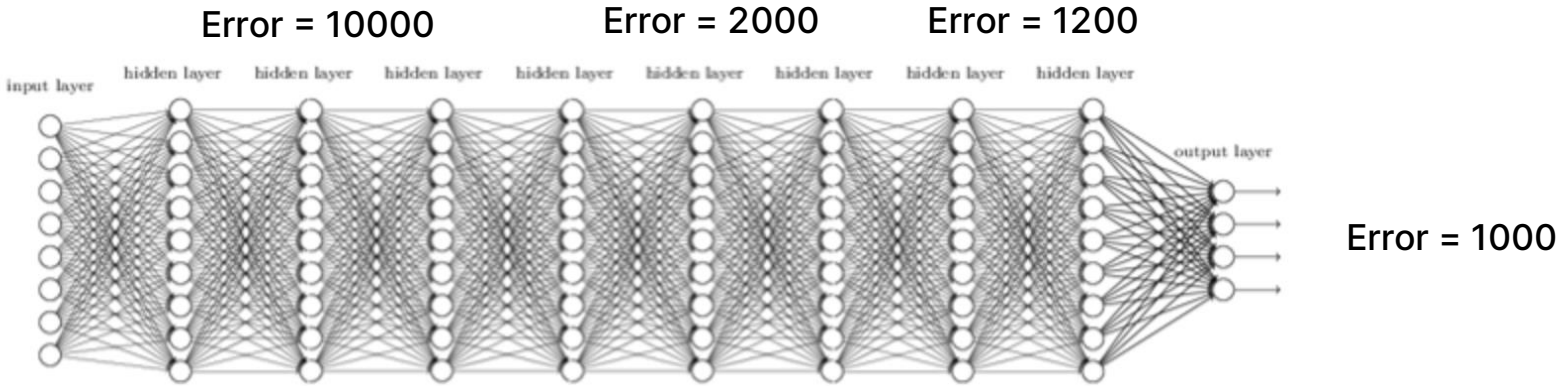
Exploding Gradient Problem

Vanishing Gradient 문제와 반대로, backward value가 폭발적으로 증가해서 모델이 제대로 학습하지 못하는 경우가 있음

BackPropagation에는 activation function의 미분 뿐만 아니라, 각 단계에서의 가중치 값들도 Gradient 전달에 관여하게 됨
따라서, 가중치 값들이 과하게 큰 경우 레이어가 깊어질수록 가중치들이 반복적으로 곱해지면서 backward value가 폭발적으로 증가하는 것



Gradient가 너무 커져서, loss가 발산하게 됨



How to Maintain Stable Gradient

그렇다면, Output layer -> Input Layer까지 Gradient를 너무 크지도 않고 / 너무 작지도 않게 Stable하게 잘 전달하는 것이 중요함

이를 위한 몇 가지 Technique를 이해해야 함

1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection – 다음 기회에
5. Gradient Clipping – 다음 기회에

등등 위에 작성되지 않은 수많은 전략이 있음.

Activation Function은 이미 학습했으니, 나머지 전략들 중 일부를 이해해봅시다

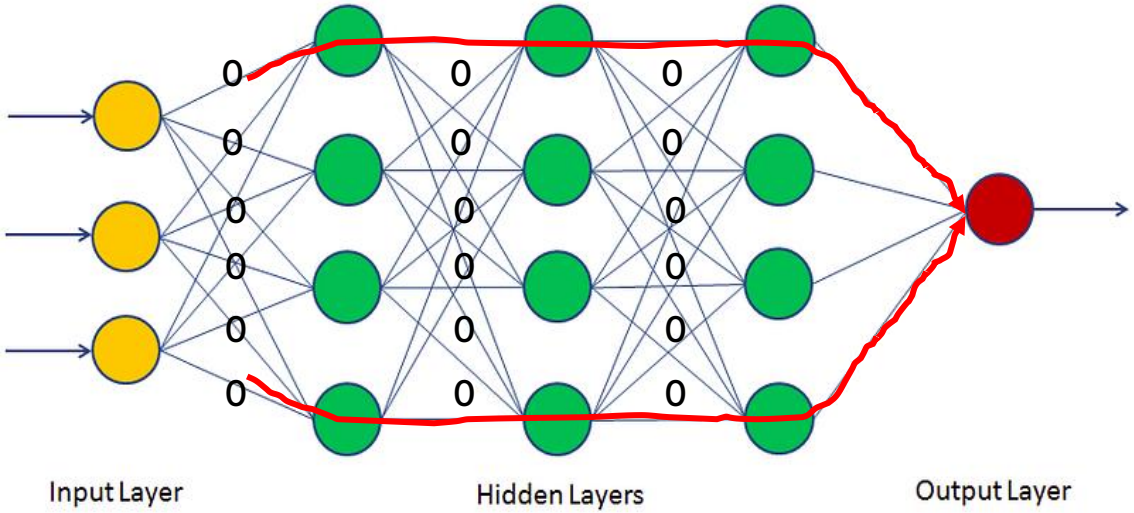
1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Weight Initialization

일단 가장 쉬운 생각으로, 모든 가중치를 0으로 초기화하고 모델로 하여금 0부터 시작해서 직접 학습하도록 하면 괜찮지 않을까 ?

결론부터 말하자면, 모든 가중치를 동일한 값으로 초기화하는 행동은 절대 절대 하면 안됨

신경망 학습의 핵심은 각 뉴런들이 서로 다른 역할을 하도록 만들어서, 표현력을 높이고자 하는 것임
그런데, 모든 가중치가 0으로 시작하면 forward 연산이 똑같아지고, backpropagation할 때 똑같은 gradient를 받아 똑같이 갱신됨



1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Gaussian Random Initialization

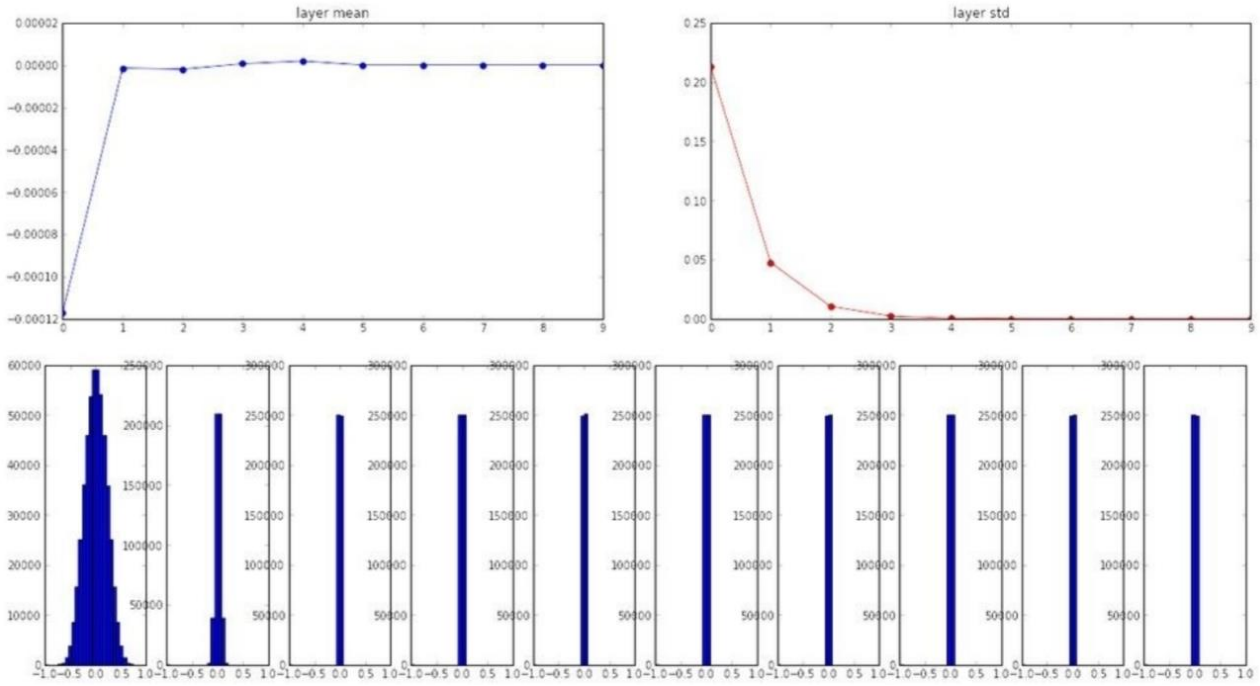
가장 쉬운 아이디어로, 가중치를 평균 0 / 표준편차가 0.01인 가우시안 분포 속에서 random하게 작은 값으로 초기화를 시켜봄

(가중치 값이 크면, Exploding Gradient 문제가 발생하니까 작은 값으로 초기화 하는게 맞음)

$$W \sim N(0, 0.01^2)$$

$$W = 0.01 * np.random.randn(D, H)$$

그리고 각 레이어 속 가중치들의 평균과 표준편차를 찍어봄



결론 평균이 0이고, 표준편차도 0에 수렴함

-> 모델의 모든 가중치가 0이 되어 버리는 상황이 발생

Gaussian Random Initialization은
network가 작을 때는 어느정도 동작하는데
DNN에서는 성능이 좋지 않음을 확인

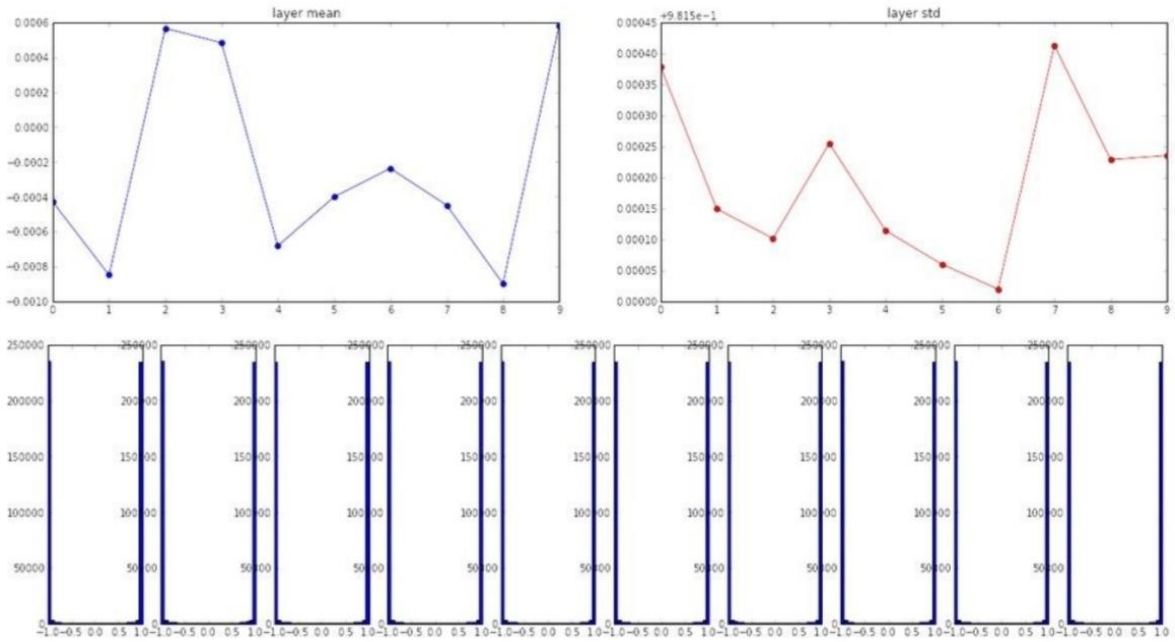
1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Gaussian Random Initialization

0.01이 문제라고 생각해서 가중치를 평균 0 / 표준편차가 1인 가우시안 분포 속에서 random하게 작은 값으로 초기화를 시켜봄

$$W \sim N(0,1)$$

$$W = np.random.randn(D,H)$$



0 아니면 1로 가중치 값이 수렴해버림

따라서 여전히 Gradient가 잘 넘어오지 않는 문제가 있음

1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

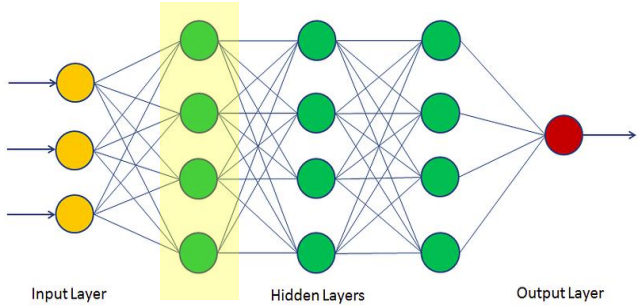
Xavier Initialization

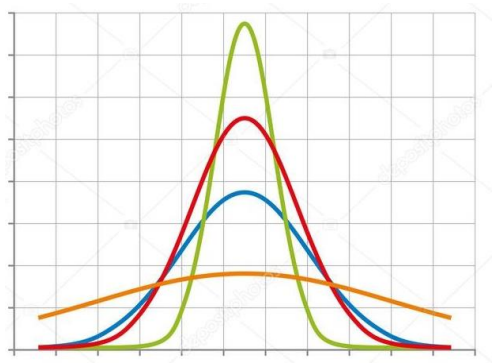
입력과 출력의 분산이 어느정도 일정하게 유지되도록 weight를 조절하자는 생각에서 나옴
레이어마다 입력과 출력 차원에 맞춰 분산을 조정하는 것

$$W \sim N(0, Var(W)) \qquad Var(W) = \sqrt{\frac{2}{n_{in} + n_{out}}}$$

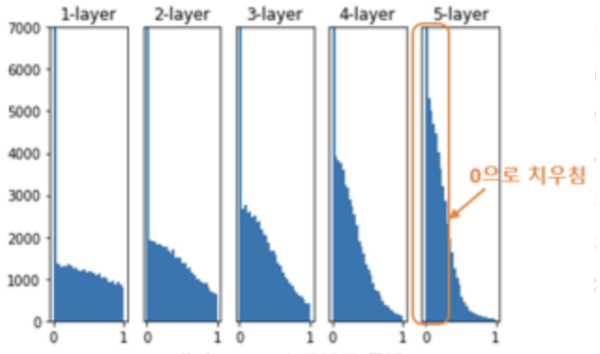
$$W \sim U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, +\sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

$n_{in} = 3, n_{out} = 4$





이렇게 레이어마다, 분포의 생김새가 달라짐.
평균 자체는 0이 맞지만, 분산이 모두 다른 것



문제는 가장 많이 쓰는 ReLU 함수랑 잘 맞지 않는다는 것
ReLU에서 출력 값이 0으로 수렴하는 문제가 종종 발생함

1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

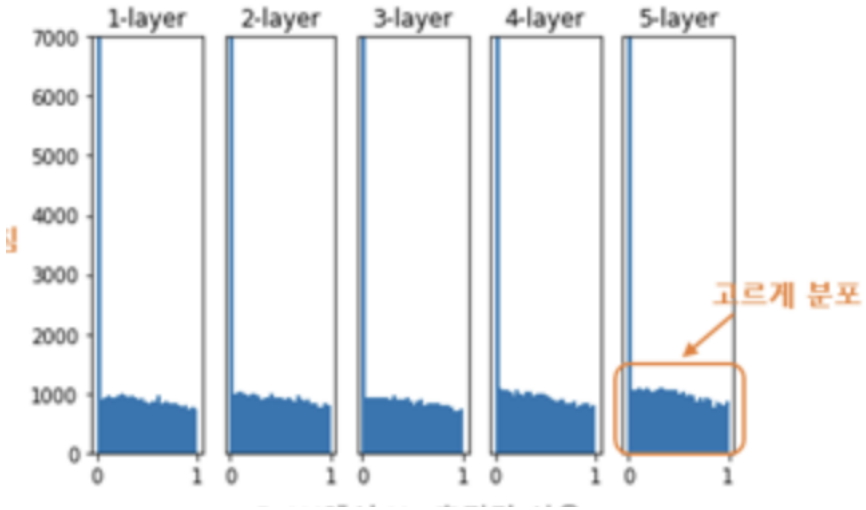
He Initialization

Xavier 초기화를 시키니까, ReLU에서 출력의 절반이 0으로 죽는 경우가 많아졌음
따라서 He initialization은 out은 고려하지 말고, in만 고려하자는 것임. 따라서 조금 더 큰 분산으로 초기화를 함

Xavier = 입력과 출력의 균형을 잘 맞추자 -> He = ReLU에서 gradient가 죽지 않도록 입력 쪽만 보고 스케일을 키워보자

$$W \sim N(0, Var(W)) \qquad Var(W) = \sqrt{\frac{2}{n_{in}}}$$

$$W \sim U(-\sqrt{\frac{6}{n_{in}}}, +\sqrt{\frac{6}{n_{in}}})$$



ReLU를 써도 고르게 분포하면서 문제가 잘 해결됨

1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Covariate Shift Problem

Batch, Layer Normalization에 대해 알기 전에 이런 개념이 왜 나왔는지에 대해서 먼저 이해해야 함

간단한 예시로, Training을 할 때는 ' 러시안 블루 ' 종으로만 사용하고, Test 시에는 ' 페르시안 ' 종의 고양이를 분류하려고 한다면 분류가 잘 안될 것임

즉, 학습 데이터와 테스트 데이터의 분포가 다른 것을 Covariate Shift Problem 이라고 부름

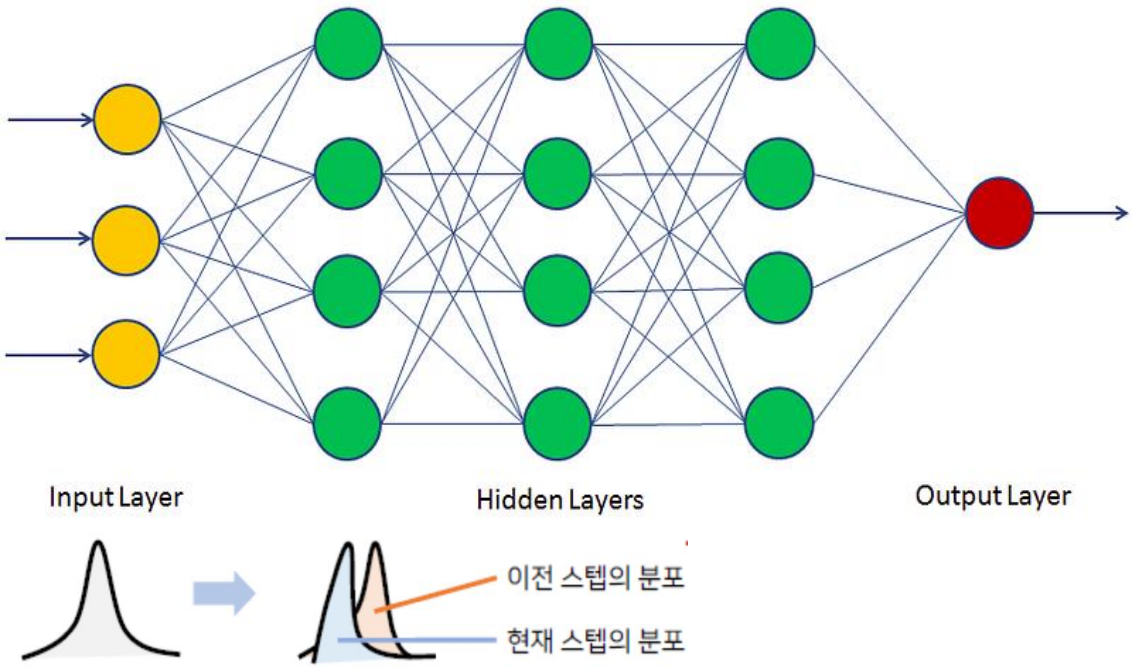
1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Internal Covariate Shift Problem

데이터의 분포가 다른 현상은 Neural Network 내부에서도 일어날 수가 있음

매 학습 step 마다, Hidden Layer에 들어오는 입력 분포가 달라지는 현상이 발생할 수 있다는 것임

결국 Hidden Layer는 우리가 알지 못하는 변환을 수행하는 녀석들인데, 이들 때문에 기존의 데이터 분포가 점점 왜곡될 수 있다는 것임
-> 앞 단의 작은 계층 변화가 점점 상위 계층으로 올라가면서 큰 영향을 미칠 수 있음

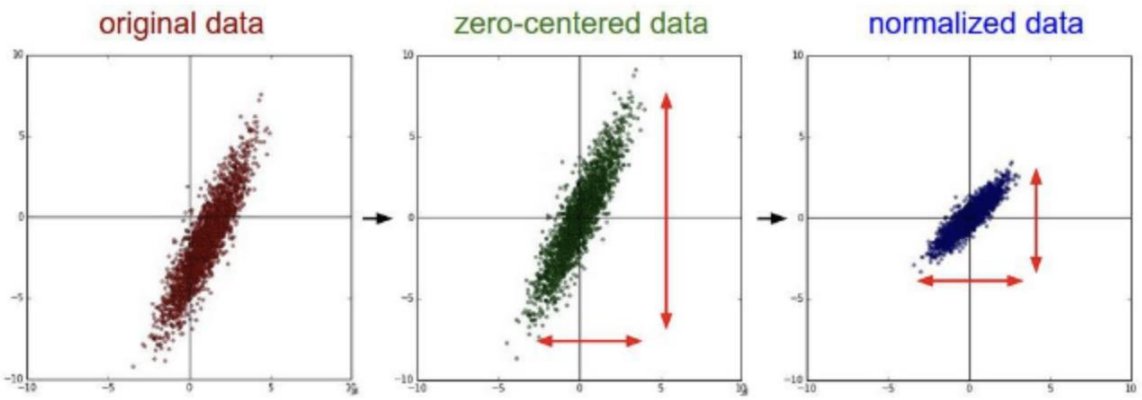


1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Normalization

Layer 마다 분포가 달라지는게 문제라면, 매번 분포를 동일하게 맞추기 위해 Normalization을 수행하면 될 것 같음

보통 쉽게 떠올릴 수 있는 Normalization에는
1. Min-Max Scaling : [0, 1] 범위 안으로 데이터를 scaling 하는 방법
2. Standardization : 평균 0, 표준편차 1인 분포로 정규화



Normalization (Standardization)

진짜 각 레이어마다 들어가는 데이터의 분포를 channel도 상관 없이 그냥 표준화 해주는 것이 좋을까?

No, 네트워크의 표현력을 제한하게 될 수 있음

1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Batch Normalization

각 레이어에 들어가는 input을 channel이나, 데이터 개수도 상관 없이 항상 표준화 해버린다면 네트워크의 표현력이 떨어짐

따라서, Batch Normalization 이라는 개념이 등장함

Batch Normalization을 제안한 논문에서 제시되어 있는 Batch Normalization을 Pseudo 코드로 확인해보면 다음과 같음

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1\ldots m}\}$;
Parameters to be learned: γ, β

Output: $\{y_i = \mathrm{BN}_{\gamma,\beta}(x_i)\}$

$$\mu_\mathcal{B} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \text{// mini-batch mean}$$

$$\sigma_\mathcal{B}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m} (x_i - \mu_\mathcal{B})^2 \qquad \text{// mini-batch variance}$$

$$\widehat{x}_i \leftarrow \frac{x_i - \mu_\mathcal{B}}{\sqrt{\sigma_\mathcal{B}^2 + \epsilon}} \qquad \text{// normalize}$$

$$y_i \leftarrow \gamma \widehat{x}_i + \beta \equiv \mathrm{BN}_{\gamma,\beta}(x_i) \qquad \text{// scale and shift}$$

Algorithm 1: Batch Normalizing Transform, applied to activation x over a mini-batch.

1. 전체 데이터에 대한 평균이 아니라 Mini-batch 별 평균과 분산을 계산한다

2. 각 배치별로 normalization을 수행한다

3. Normalization 이후에 2개의 파라미터를 사용해서 모델의 표현력을 유지한다

γ = 출력 분포의 "폭"을 조절 / β = 출력 분포의 "중심"을 조절

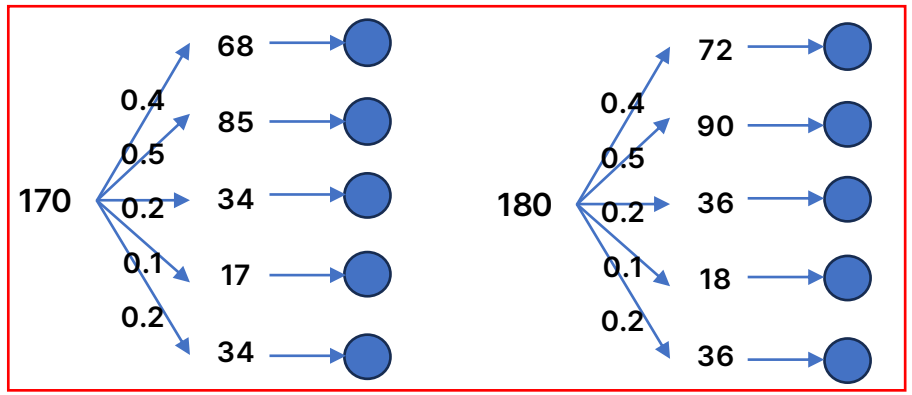
무조건 평균을 0 / 분산을 1로 만드는 것이 아니라, 학습 가능한 파라미터를 둬서,
최종 분포를 원하는 모양으로 찾아감으로써 표준화 + 표현력 유지 동시에 달성

1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

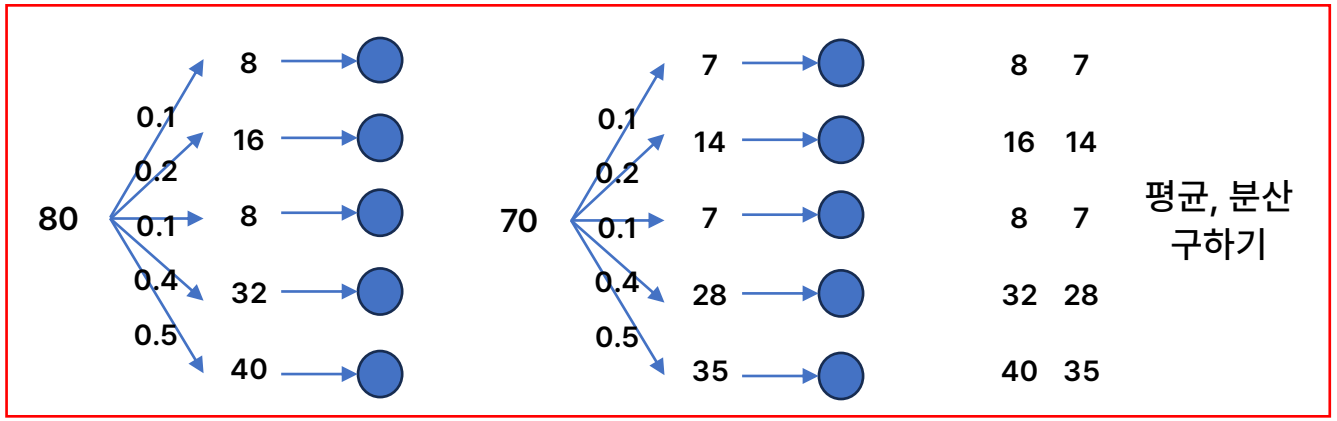
Batch Normalization

근데 그냥 무작정 Normalization 하는게 아니라, 들어온 녀석들 중 같은 channel의 데이터끼리 평균, 분산을 구해서 normalization 하는 것임

2번 차원(키)의 평균, 분산 구하기



1번 차원(체중)의 평균, 분산 구하기

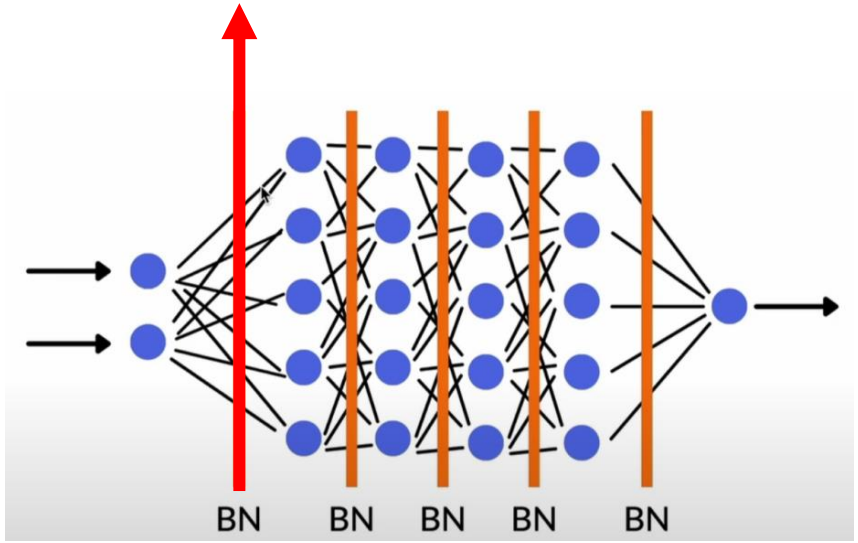


	1번 차원	2번 차원
	체중	키
1번 데이터	80	170
2번 데이터	70	180
3번 데이터	80	198
4번 데이터	70	175
…		
100번 데이터	73	181

Batch 1

Batch 2

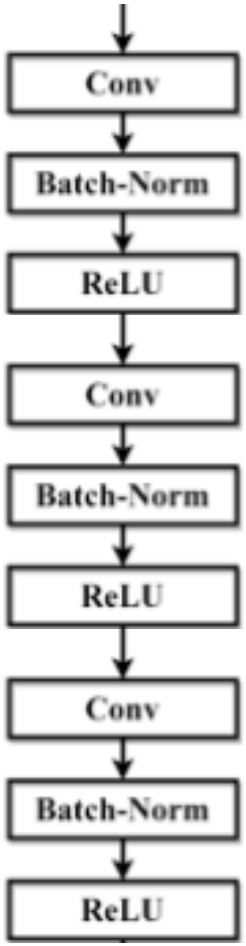
80	70	체중
170	166	키



1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Batch Normalization

따라서 실제 모델에서는 보통 이렇게 각 layer 마다 Batch-Normalization을 끼워둔다는 사실



근데 학습은 Batch 단위가 있으니까, 들어오는 Batch 마다 평균, 분산을 구하면 되는데

Inference 할 때는 하나씩 예측을 하는 건데..?

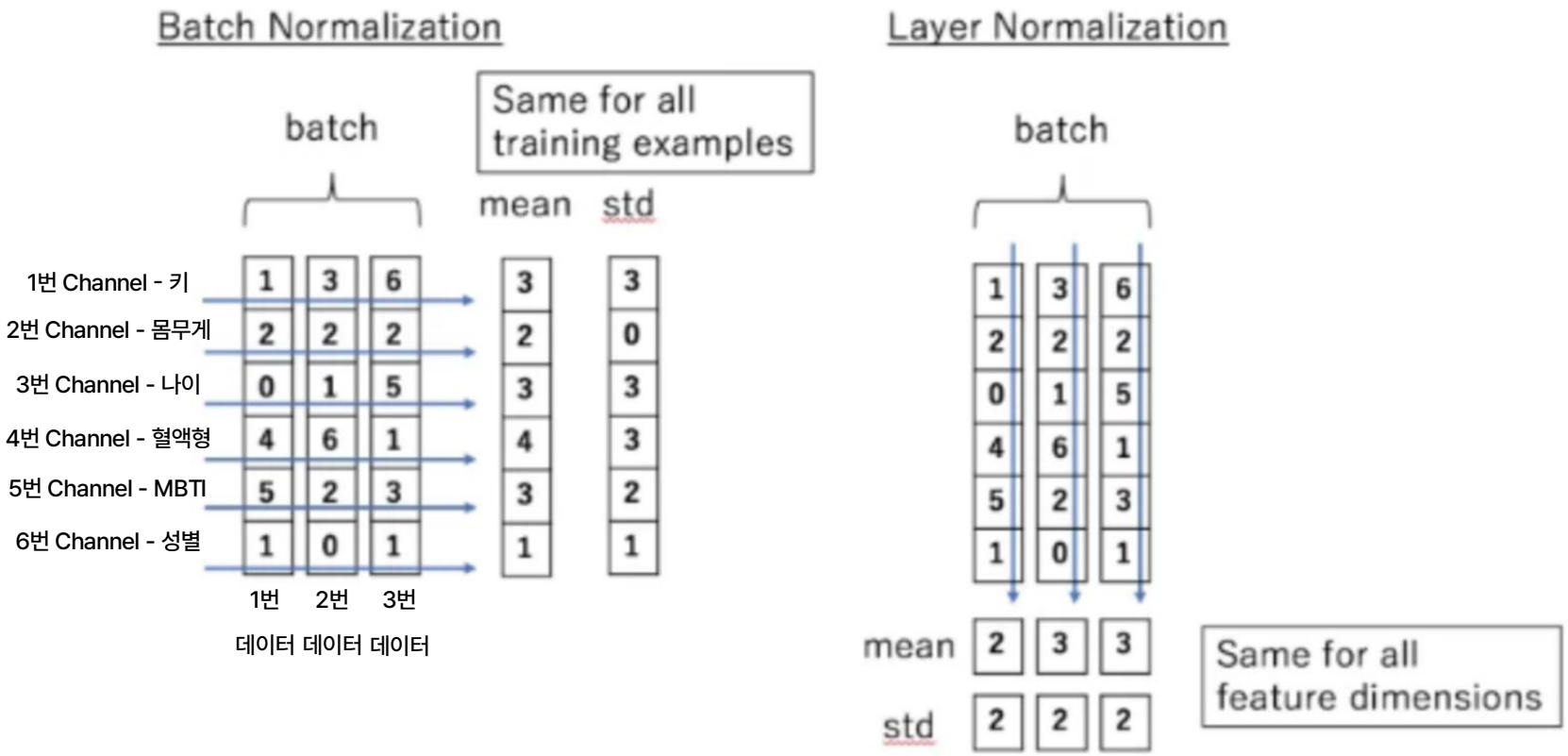
따라서 Inference 할 때에는 전체 데이터의 평균, 분산을 구하고 그걸 사용함

1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Layer Normalization

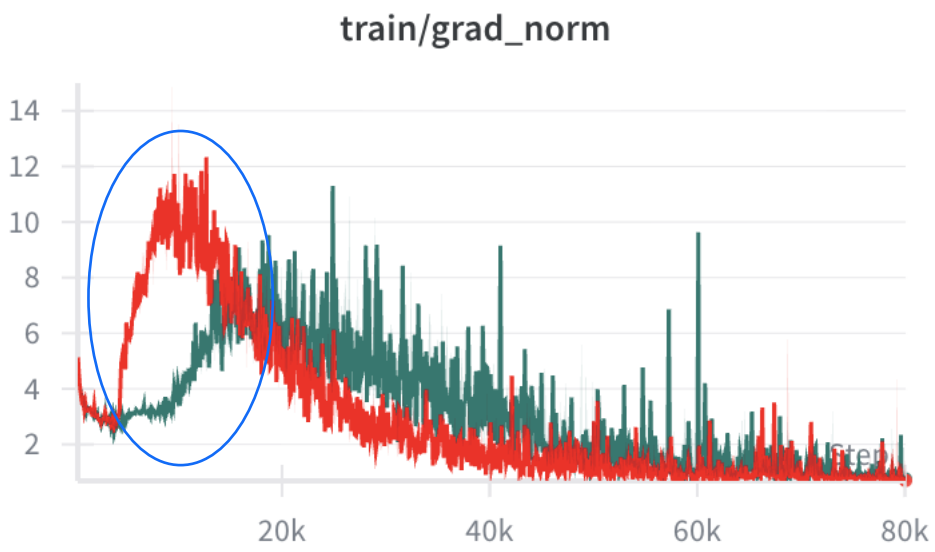
Batch Norm은 batch size가 작거나, 데이터 별 편차가 큰 경우에 극단적인 결과를 내지만
Layer Normalization은 각 데이터 별로 normalization을 수행하는 것이기 때문에 작은 batch size에서 효과적임

또한 sequence 데이터를 처리하는 경우, batch norm을 적용하기 어려운데, Layer는 데이터 단위로 보기에 RNN 에서도 효과적임



1. Activation Function
2. Weight Initialization
3. Normalization
4. Residual Connection
5. Gradient Clipping

Experiment



실제로 학습시키는 모델에서, Gradient 흐름을 찍어보았음

Image

What is Image?

지금까지 배운 모델들의 입력은 항상 csv로 정리되어 있는 데이터였음

보통 row와 column으로 깔끔하게 정리되어 있는 2차원 데이터를 분석에 사용했음 [H, W]

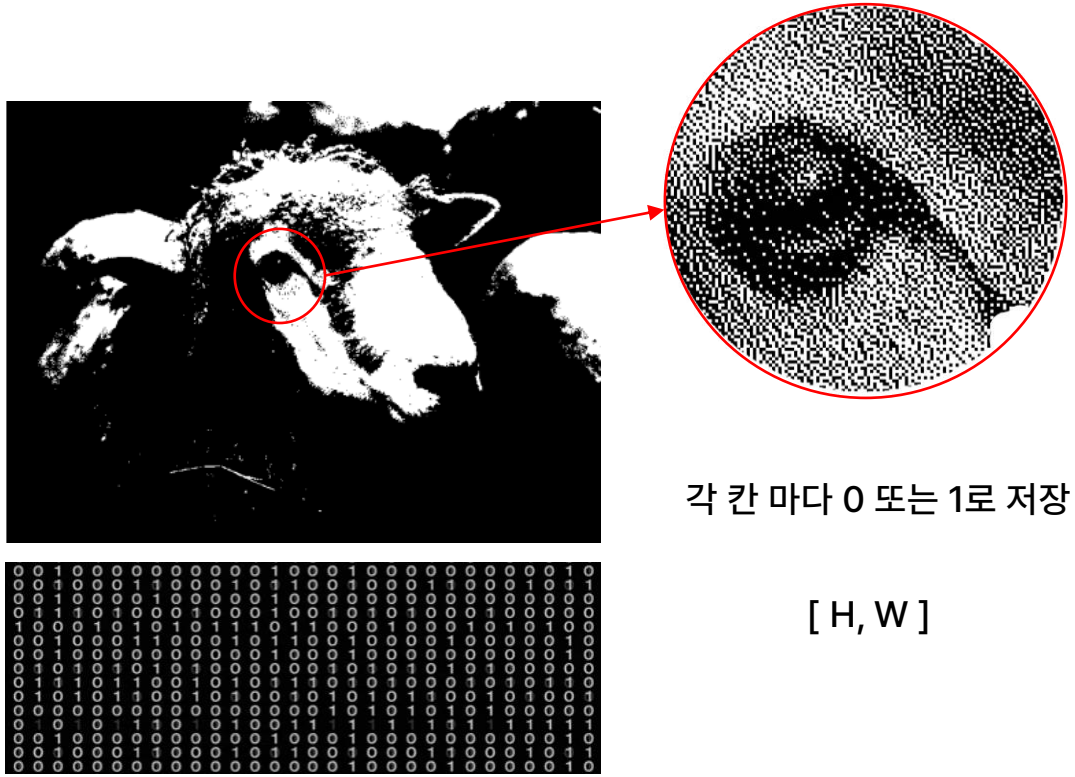
그렇다면 이미지는 도대체 어떻게 생겨먹은걸까 ?

날짜	유저ID	전화번호	매출	비용
2024-01-02	1	010-1234-5671	20000	2000
2024-01-02	2	010-1234-5672	40000	3000
2024-01-02	3	010-1234-5673	10000	1000
2024-01-02	101	010-4321-5671	120000	0
2024-01-02	102	010-4321-5672	150000	0
2024-01-02	103	010-4321-5673	50000	0

What is Image?

이미지도 한 종류만 있는게 아니라 여러 가지 종류가 있음

1. Binary scale Image
2. Grayscale Image
3. RGB, color Image

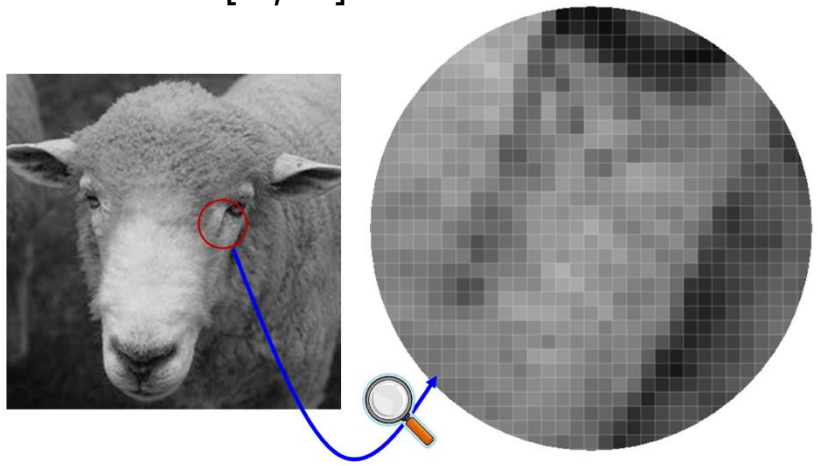


각 칸 마다 0 또는 1로 저장

[H, W]

각 칸 마다 0 ~ 255 로 저장

[H, W]



147	146	148	150	153
145	149	151	154	156
149	152	153	156	157
150	153	155	157	158
149	151	152	156	159

RGB color Image

그 중에서 가장 많이 사용되고, 보통의 이미지 데이터라고 볼 수 있는 RGB Color Image

각 픽셀은 red, green, blue의 양으로 묘사됨

각 색상의 요소는 0 ~ 155 범위를 가지기에, $2^8 = 256 \rightarrow 2^8 * 2^8 * 2^8 = 16{,}777{,}216\ colors$ 까지 표현할 수 있는 것임

R

217	216	218	220	223
215	219	221	224	226
219	222	223	226	227
220	223	225	227	228
219	221	222	226	229

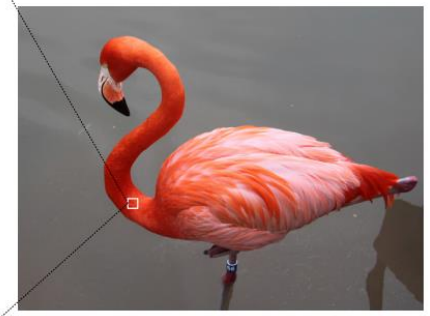
G

57	56	58	60	63
55	59	61	64	66
59	62	63	66	67
60	63	65	67	68
69	61	62	66	69

B

37	36	38	40	43
35	39	41	44	46
39	42	43	46	47
40	43	45	47	48
39	41	42	46	49

typically 24 bit / pixel
(= 16,777,216 colors)



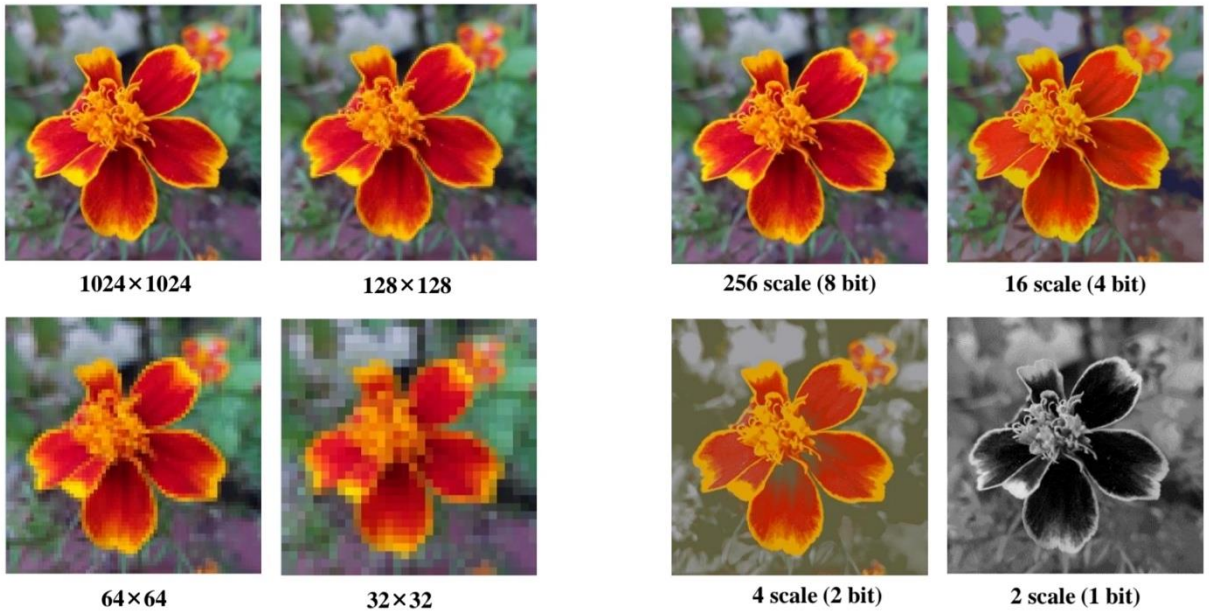
Resolution (해상도)

그렇다면 이미지는 도대체 어떻게 저장되어 있는거지 ? 를 알기 위해선 Resolution의 개념을 가지고 있어야 함

결국 하나의 이미지가 몇 개의 칸으로 구성되어 있는지를 의미하는 것임

아래 그림에서 알 수 있듯이, 똑같은 크기의 캔버스 위에 사진을 올려본다고 했을 때, resolution이 높으면 높을수록 훨씬 더 선명해 보임
너무 당연한 것, 같은 사진을 얼마나 더 세세하게 나눴는지를 말하는 것임

따라서 해상도가 높을수록 선명해지지만, 그만큼 담고 있는 정보가 많으니까 데이터의 용량이 커진다는 것임 → 한 칸당 csv 한 칸과 동일함



Resolution (해상도)

아 내 갤러리에 사진이 너무 많아.. 왜 이렇게 저장 공간을 많이 잡아먹는거야 ?

1024×1024

칸 개수 = 1024 x 1024개 = 1,048,576개

Grayscale = 1칸에 256개의 값을 가질 수 있음 = 1 byte ($2^8 = 1{,}048{,}576$)

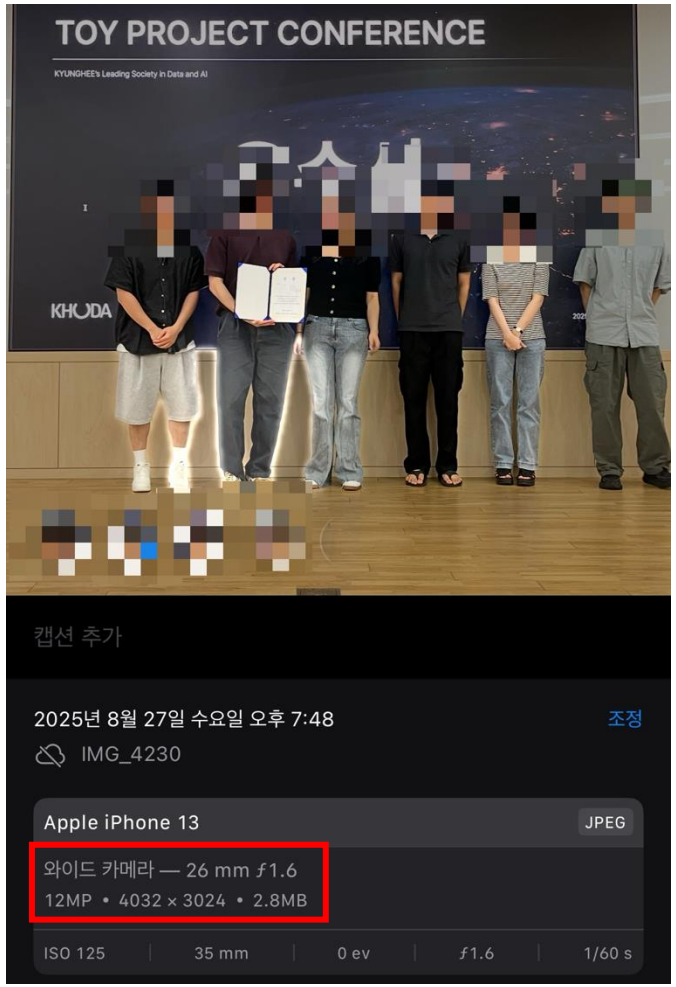
저장 용량 = $1{,}048.576 * 1\ byte = 1{,}048{,}576\ bytes \approx 1.0\text{MB}$

1024×1024

칸 개수 = 1024 × 1024개 = 1,048,576개

RGB = 한 칸이 사실 뒤로 2칸 더 숨겨져 있어서, 총 3칸임 = 3 bytes ($2^8 = 1{,}048{,}576$)

저장 용량 = $1{,}048.576 * 3\ bytes = 3{,}145{,}728\ bytes \approx 3.0\text{MB}$



근데 고작 100만 화소에 만족하실 수 있나요 ?

총 1200만 픽셀로 저장된 이미지 (2.8MB 인 이유는 불필요한 정보를 제거하면서 압축해서 저장하기 때문)

Image Processing

$$1{,}048.576 \ * 3\ bytes = 3{,}145{,}728\ bytes \approx 3.0\text{MB}$$

고작 3MB 짜리 이미지도 픽셀이 100만 개가 넘어가는데, 이걸 학습시키려면 input node가 100만 개가 필요하다는거야 ?

(물론, 고차원 tensor에 대해서 굳이 1열로 쭉 나열하지 않고 연산을 바로 해버리긴 하는데, 아무튼 이해를 위해서)

1999년 - IBM이 170MB 및 340MB 용량의 Microdrive 를 출시했습니다.

HDD가 1999년도나 되어서야 340MB 용량을 달성했는데, 모든 노드를 연결하는 딥러닝을 이미지에 적용하는 것은 상상도 하지 못했음

그럼에도, 이 당시에 이미지를 다루는 머신러닝 방법이 있긴 했으니

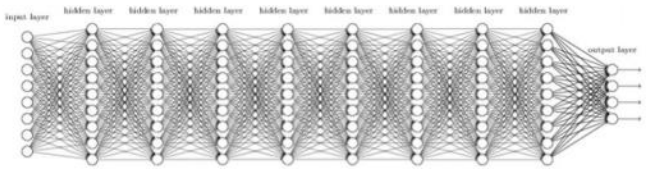
그 방법이 바로 이미지에서 직접 feature를 추출하고, 추출된 feature에 머신러닝을 적용하는 방법

초창기 Image Processing 분야에 호기심이 있는 분들은 아래 링크를 참고)
https://blog.naver.com/PostView.nhn?blogId=dongju0531hb&logNo=222443993008
https://learnopencv.com/image-recognition-and-object-detection-part1/

Pytorch

Pytorch

	CPU	GPU
명세	Intel® Core™ i9-13900K Processor	NVIDIA GeForce® RTX™ 4090
Base Clock Frequency	3.00GHz	2.23GHz
일꾼 머리 수 Cores	24(32threads)	16384
Memory Bandwidth	89.6GB/s	1,008GB/s
Floating-Point Calculations	845GFLOPS	82.58TFLOPS
Cost	~$600.00	~$1600.00



CPU는 교수님 정도의 일꾼을 소규모 거느리는 것
GPU는 초등학생 정도의 일꾼을 대규모로 거느리는 것

근데 딥러닝은 일꾼의 퀄리티 보다는 일꾼의 수가 절대적으로 중요한 분야임
(이건 너무 쉬운 설명)

Pytorch

	CPU	GPU
명세	Intel® Core™ i9-13900K Processor	NVIDIA GeForce® RTX™ 4090
Base Clock Frequency	3.00GHz	2.23GHz
일꾼 머리 수 Cores	24(32threads)	16384
Memory Bandwidth	89.6GB/s	1,008GB/s
Floating-Point Calculations	845GFLOPS	82.58TFLOPS
Cost	~$600.00	~$1600.00

핵심은 데리고 있는 일꾼을 쉽게 병렬로 쓸 수 있다는 것임

자아를 가진 사람이 많으니까, 그들에게 서로 다른 연산을 동시에 딱 시키는 것

10000개의 문제가 있을 때,

CPU는 교수님 24명이 순차적으로 하나씩 하나씩 계산해나감
GPU는 동시에 10000개의 문제를 1개씩 가져가서 풀어버림

또한, **Memory Bandwidth** - 메모리와 데이터 주고받는 속도 **(GPU**가 훨씬 넓음 → 딥러닝에 중요**)**

근데 GPU 한테 일을 어떻게 시키지..?

Pytorch

오케이 GPU의 중요성은 알겠어, 근데 Pytorch는 왜 배우는건데 ? 그게 뭔데 ?

```
x = torch.randn(3, 3)              # CPU에 텐서 정의
x = x.to("cuda")                   # GPU로 이동 (매우 간단)
y = torch.randn(3, 3).to("cuda")   # GPU에 있는 또 다른 텐서
z = x + y                          # GPU에서 연산됨
```

Pytorch를 배우는 가장 큰 이유는 CUDA를 쉽게 쓰기 위함임

1. CUDA란?
- CUDA = NVIDIA가 만든 GPU 프로그래밍 언어/플랫폼
- C/C++/Fortran 같은 저수준 코드로 GPU 코어에 직접 연산을 날릴 수 있음
- 문제는, 이걸 직접 쓰려면 GPU 아키텍처, 메모리 관리, 스레드 동기화까지 다 알아야 해서 너무 복잡함

2. Torch란?
- 2002년경 Lua라는 언어 위에서 만들어진 머신러닝/수치 연산 프레임워크
- CUDA 바인딩을 제공 → GPU 연산 가능 / NumPy 같은 텐서 연산 지원 / 신경망 모듈 (nn 패키지) 내장

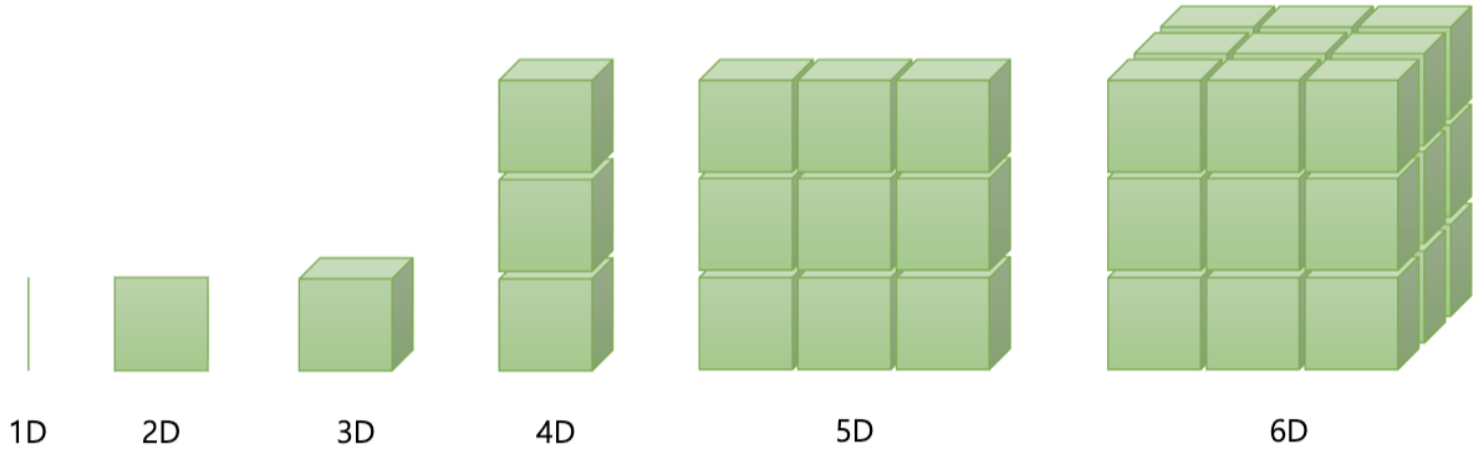
문제는 Lua 언어가 인기가 너무 없어서 FaceBook AI 팀에서 torch의 아이디어를 python으로 다시 구현함 -> Pytorch

3. PyTorch란?
- CUDA 위에 올라가는 추상화 계층
- tensor.cuda() 이 한 줄로, 데이터나 모델이 GPU 메모리에 올라가고 병렬 연산이 실행되는 것임
- Scratch로 GPU를 쓰기 위한 코드들이나, 미분 등 다양한 연산을 위한 코드를 직접 구현하지 않아도 되는 것

Tensor

Tensor란 ?
- 파이토치의 데이터 형태 (array, 넘파이 등등 이런 느낌)
- 단일 데이터 형식으로 구성된 자료들의 다차원 행렬

어떤 데이터를 사용해서 딥러닝 학습을 시키건, Tensor 형태로 GPU에 올라가서 학습되는 것임



이미지 데이터는 보통 어떻게 생겼을까?

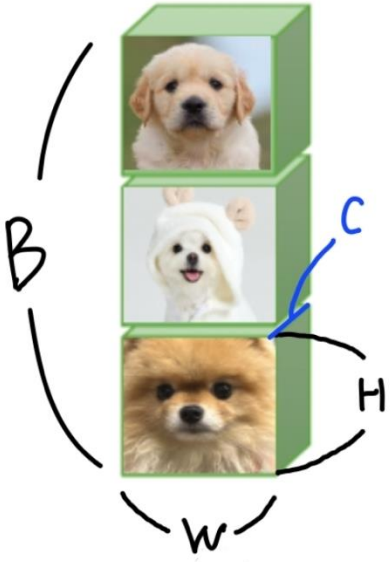
Tensor

1D

만약, Batch size를 1로 가져가면 1D tensor(Array)가 생겨날 수 있음
1장의 [H, W] 이미지를 1열로 나열시켜서, 그걸 Input Node에 각각 다 연결하면 이런 식의 input이 생겨날 수 있음

2D

1장의 Grayscale / Binary scale Image가 이런 식으로 생겼음
[H, W] 각 픽셀에 값이 하나씩 뿌려져 있으니까



3D

1장의 RGB Color Image가 이런 식으로 생겼음
Red / Green / Blue 세 색상마다 [H, W] 각 픽셀에 값이 하나씩 뿌려져 있으니까 3채널(RGB) [3, H, W]

Input Layer 관점에서의 설명임
Hidden layer를 거치면서 feature가 되니까, 3 channel이 아니라 수십, 수백 channel이 됨

4D

보통 우리가 보는 이미지 데이터는 다음과 같이 처리됨
한 장 당 [C, H, W] 이런 것이 보통 Batch 단위로 처리되니까 [B, C, H, W] 데이터가 이렇게 4D tensor 형태로 생겼음

[B, C, H, W] 이미지랑, 각 이미지별로 가중치들이 또 tensor 형태로 정리되어 있어서, (tensor 곱하기 tensor) 이게 바로 가능함